

BTF Trace Viewer

Version 1.4.0 — Desktop and Web

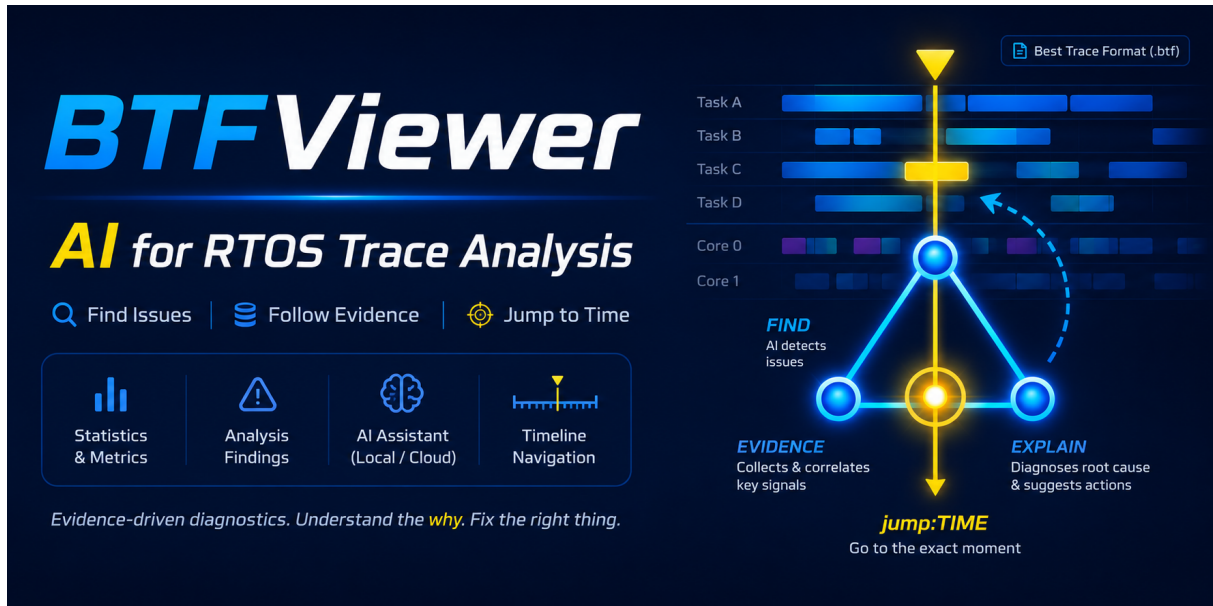


Figure 1: BTFViewer AI-assisted analysis

BTFViewer turns BTF traces into verifiable multicore scheduling evidence—with deterministic statistics, before-and-after comparison, CI-ready reports, and optional AI-assisted investigation.

BTFViewer analyzes recorded context-switch activity from real-time operating systems (RTOSs) in **Best Trace Format** (.btf). It is designed for post-capture analysis on Desktop or Web and complements low-level debuggers and target-side trace recorders. It does not read source code or ELF files, or simulate an RTOS scheduler; its conclusions remain tied to events recorded in the trace.

BTF Trace Viewer

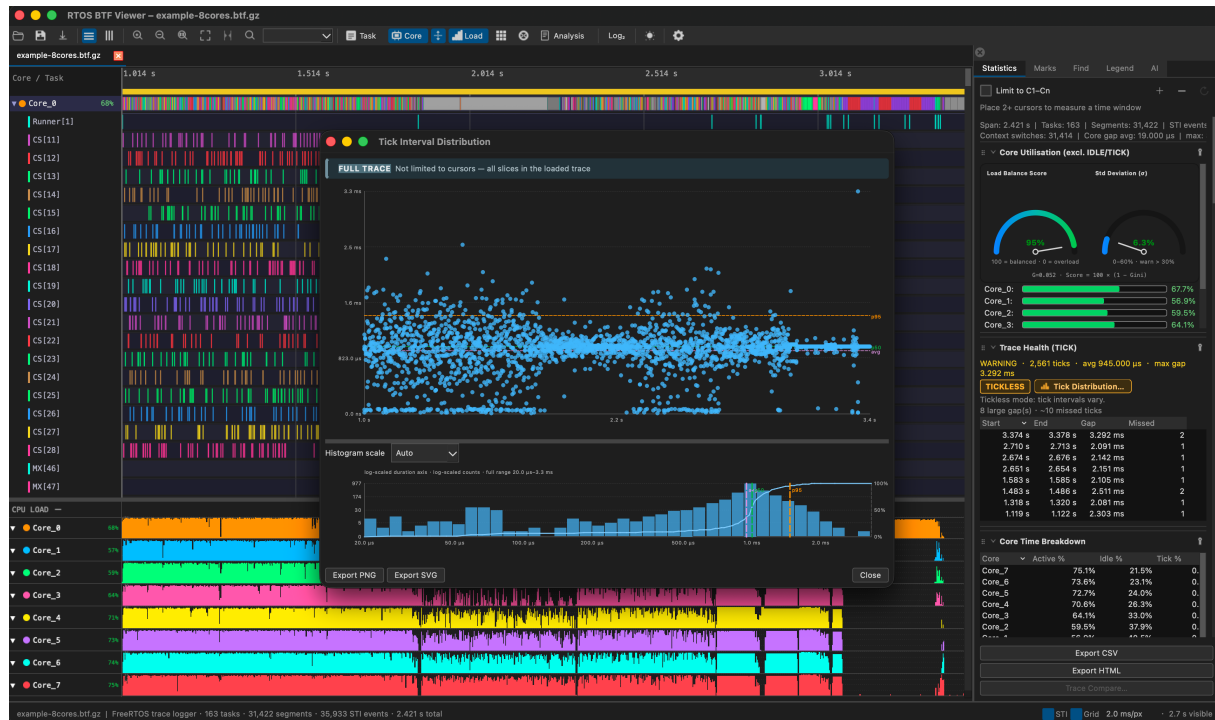


Figure 2: BTFViewer

[Try the live demo](#)

Why BTFViewer?

BTFViewer is designed to shorten the path from a timing symptom to evidence that can be reviewed and reproduced.

- **Evidence you can verify:** Statistics and Analysis Findings are calculated from recorded BTF/STI events. Metric definitions, limitations, and timeline locations remain visible, so every conclusion can be checked against the trace.
- **Multicore behavior made explicit:** Dedicated diagnostics reveal load imbalance, concurrent core activity, task affinity, migration frequency, ping-pong movement, and hot core-to-core migration paths—not just which task ran when.
- **Comparison built into the workflow:** Open multiple traces, measure before-and-after deltas and distribution changes, save baselines, and use the Desktop CLI as a regression gate in automated tests or CI.

- **AI grounded in measured data:** The optional AI Assistant can triage, investigate, verify, evaluate heuristic experiments, and explain trace comparisons. Statistics and the timeline remain the source of truth; a local OpenAI-compatible endpoint can keep extracted evidence on the local machine.
- **Easy to run and share:** Desktop and Web use the same analysis workflow. The standalone Web build and bilingual guided demo make it easy to review or demonstrate a trace without a specialized analysis installation.

Features

Area	Capabilities
Timeline and measurement	Task or CPU-core views, horizontal or vertical layouts, zoom, pan, search, cursors, bookmarks, and annotations
Statistics and diagnostics	Utilisation, execution, blocking, dispatch latency, jitter, preemption, migration, mutex, semaphore, queue, deadline, and anomaly analysis
Trace comparison	Multi-tab sessions, before-and-after deltas, distribution comparisons, saved baselines, and experiment validation
AI Assistant	Finding triage, scoped investigation, evidence verification, heuristic what-if analysis, and comparison explanations
Export and automation	PNG, SVG, CSV, HTML, Perfetto, selected BTF ranges, and Desktop CLI support for scripts and CI
Learning and sharing	Standalone Web application and an 8-core guided demo with English or Traditional Chinese narration

Documentation

Document	Purpose
README.md	Install BTFViewer and learn its main functions
WORKFLOWS.md	Follow step-by-step investigation procedures
STATISTICS.md	Understand metric definitions, formulas, and interpretation
AI.md	Configure and use AI-assisted investigation
demos/README.md	Create and maintain guided demos

If you are new to BTFViewer, begin with [Quick start](#), then follow the [Basic analysis workflow](#). Refer to the other documents for detailed procedures and metric definitions.

Quick start

Desktop application

Requirements:

- Python 3.8 or later. Python 3.9 or later is recommended.
- PySide6 6.4 or later. The command below installs it with the other dependencies.

```
cd BTFViewer
pip install -r requirements.txt
python builds/btf_viewer.py trace.btf
```

Replace `trace.btf` with the path to the trace file. If no file is specified, BTFViewer restores the previous session. Files can also be opened through **File → Open**, **File → Open Recent**, or drag and drop.

Web application

Open the standalone file in a modern browser:

```
open BTFViewer/builds/btf_viewer.html
```

BTF Trace Viewer

You can also double-click the file or use the [hosted demo](#). A local server is usually not required.

To rebuild the Web application:

```
cd BTFViewer
make web
```

Supported files

Format	Description
.btf	Best Trace Format
.btf.gz, .gz	Gzip-compressed trace
.btf.bz2, .bz2	Bzip2-compressed trace
.btf.zip, .zip	Archive containing one or more BTF traces; each trace opens in a separate tab
.xml	Demo script used by the Web application
.xtf	Portable demo package containing a script, trace, and optional narration

Sample traces are available in `tracedata/`, including `example-2cores.btf.gz`.

Guided demo

The `demo_8cores` package presents the main workflow with an 8-core trace and spoken instructions. Run it before opening an application trace to become familiar with the interface and analysis sequence.

Run the demo

Desktop:

```
cd BTFViewer
make demo
make demo DEMO_LANG=zh-tw
```

You can also open either package directly:

```
python builds/btf_viewer.py demos/demo_8cores/demo_8cores.xml
python builds/btf_viewer.py builds/demo_8cores.xtf
```

Web:

- Select **Demo** on the toolbar to load the bundled tour.
- Open or drag `demo_8cores.xtf` into the viewer.
- Open the demo XML file and select its package folder when requested.

English is the default narration language. Select another language from the demo bar **Voice** menu. Press **Space** to pause or resume. Press **Esc** twice within 2.5 seconds to stop.

Build a portable demo package

```
make demo-pack
make demo-pack DEMO_LANGS=en,zh-tw
make demo-pack DEMO_LANGS=all
python3 scripts/demo_pack.py demos/demo_8cores --list-voices
```

The generated `builds/demo_8cores.xtf` contains the script, trace, and selected voice files. Open it in either the Desktop or Web application.

The Web **Record** function uses browser display capture. Select the current tab and enable tab audio to include narration.

See [demos/README.md](#) for package layout, voice generation, recording, XML actions, and the demo API.

Viewer controls

The Desktop and Web applications use the same main workflow. Platform-specific differences are noted below.

Main controls

Control	Purpose
Open	Open a BTF trace or demo package
Task / Core	Show one row per task or group activity by CPU core
Horizontal / Vertical	Change the direction of the time axis
Zoom in / Zoom out / 1:1 / Fit	Adjust the visible time range
Load	Show or hide the CPU-load chart
Heatmap	Inspect task migration and core movement
Analysis	Open automatically generated findings for the current scope
Compare	Compare two or more open traces
Find	Search for tasks, events, migrations, intervals, or synchronization objects
Settings	Configure display, layout, cursors, and AI options

The Web toolbar also provides **Demo**, **Record**, and **About**. Some controls move into **More** when the window is narrow.

Task and core views

View	Use
Task View	Follow each task across all cores
Core View	See which task ran on each core; cores can be expanded or collapsed

Task View is useful for checking execution and migration. Core View is useful for checking utilization, idle periods, and load distribution.

Zoom, labels, and highlights

- Use the mouse wheel to pan. Hold **Ctrl** while scrolling to zoom.
- Hold **Shift** while scrolling to change the pan axis.
- Use a trackpad pinch gesture to zoom on macOS.
- Middle-drag across the timeline to zoom into a selected time range.
- Select **Fit** or press Ctrl+0 to show the complete trace. **Range** / Ctrl+R fits the time between the first and last cursor (C1-Cn). In a demo script, <zoom_view/> uses Fit, while <fit_view/> uses Range when cursors are present. **Zoom out** stops at Fit and is unavailable while the complete trace is visible.
- Select **1:1** to return to the configured zoom density.
- Click a task label or legend entry to keep that task highlighted.
- Hover over a timeline segment to view its duration, core, and nearby activity.

CPU load

Select **Load** to display the utilization chart below the timeline. Drag the divider to resize it.

When a task is locked as the current highlight, Task View shows its utilization on each core. Use this display to check whether the work is distributed as expected or moves between cores too often.

Cursors and time ranges

Cursors mark timestamps and define a measurement range. BTFViewer supports four cursors by default; the maximum can be changed in **Settings**.

Action	Method
Place a cursor	Click the timeline, or press C to place one at the viewport center
Move a cursor	Drag its line
Remove a cursor	Click near the line or use the context menu
Clear all cursors	Press Shift+C or Shift-right-click
Snap to an event boundary	Shift-click

Place at least two cursors around the period you want to inspect. Statistics can then limit calculations to the time between the first and last cursor. **Zoom to cursor range** (Ctrl+R) displays only this period. **Save selection as BTF** exports it as a smaller trace.

Bookmarks, annotations, and search

Tool	Purpose
Bookmark	Add a named timestamp
Annotation	Add a note at a timestamp
Find	Locate tasks, migrations, STI events, intervals, lifecycle events, and synchronization objects

Use Ctrl+F to open Find. Use F3 and Shift+F3 to move between results. Right-click the timeline to add or edit cursors and marks.

Multiple traces

Each trace opens in a separate tab and keeps its own zoom, cursors, marks, and filters.

- Ctrl+Tab: next tab
- Ctrl+Shift+Tab: previous tab
- Ctrl+W: close the current tab

Desktop restores files from their original paths. Web can restore up to eight packed traces from browser storage. Private browsing, storage limits, or cleared site data can prevent restoration.

Basic analysis workflow

For an initial review, use the following sequence:

1. Open the trace and select **Fit** to view its complete duration.

2. Select **Load** and check whether all cores carry a reasonable share of the work.
3. Open **Analysis** and review the highest-severity findings.
4. Open the Statistics section named by a finding.
5. Select a high value or outlier to jump to the corresponding timeline event.
6. Place cursors around the affected period and limit Statistics to that range.
7. Inspect the task, core, preemption, blocking, synchronization, or migration details.
8. If needed, ask the AI Assistant to explain or verify the measured evidence.

Start with measured evidence instead of an assumed cause. Confirm the behavior in the timeline and Statistics before drawing a conclusion. See [WORKFLOWS.md](#) for detailed procedures.

Analysis and Statistics

BTFViewer calculates all results from recorded BTF events. It does not inspect source code or ELF files, simulate an RTOS scheduler, or estimate data that is not present in the trace.

Choose the first check

Symptom	Start here	Check next
Unknown issue	Analysis Findings	Statistics section named by the finding
Tick jitter or tickless behavior	Trace Health (TICK)	Execution-time outliers
Uneven SMP load	Core Utilisation	Concurrent active cores, then migrations
High scheduler cost	Kernel Switch Overhead	Core time breakdown
Slow task execution	Execution Time	Preemption and mutex activity

Symptom	Start here	Check next
Long wait time	Blocking Time	Mutex owner and preemption activity
Ready-to-run delay	Dispatch Latency	Blocking and preemption
Priority inversion	Priority Inheritance	Mutex pairing and blocking
Frequent core movement	Core Migrations	Load balance, migration heatmap, and mutex bounces
Lock or queue issue	Mutex / Semaphore / Queue	Blocking and migrations

Detailed metric definitions and formulas are in [STATISTICS.md](#).

Analysis Findings

Select **Analysis** to review possible issues in the current trace or cursor range. Findings can include load imbalance, execution-time hotspots, blocking, priority inversion, frequent core migration, deadline misses, tick-health problems, and synchronization movement between cores.

Each finding includes a severity, a related Statistics section, and a relevant time range when available. Select a finding to open its statistical evidence, place cursors, show the range on the timeline, start an AI-assisted investigation, or save the findings as text.

For multi-core traces with measurable utilization, the core-balance finding reports a **Load Balance Score** with supporting distribution values. A high score means work is distributed more evenly. Review the timeline and migration data before deciding whether the distribution is suitable for your workload.

Reading Max, p95, and p99

- **Max** is the largest measured value. Use it to locate the worst observed event.

- **p95** is the value that 95% of samples do not exceed. It shows behavior during the slower part of normal operation without being dominated by one rare event.
- **p99** is the value that 99% of samples do not exceed. Use it to identify severe but recurring latency that an average may hide.

p95 is important because an average alone does not describe real-time performance. A good average can still hide repeated slow events. Compare p95 with p99 and Max to distinguish typical tail latency from less frequent extremes.

Core migration checks

Check load balance before interpreting migration counts. An SMP scheduler may move tasks to idle cores to distribute the workload, so some migration is expected.

After confirming load balance, check whether a task moves between cores more often than needed. Frequent migration can increase L1 cache misses. On Xtensa processors, migration can also reduce the benefit of lazy context switching: coprocessor registers may need to be saved when a task moves to another core, increasing context-switch overhead.

Review Task View, per-core load, **Core Migrations**, and the migration heatmap together. A high migration count is significant when it coincides with poor cache behavior, greater context-switch overhead, higher latency, or unstable load distribution.

Comparing open traces

Compare is available when two or more traces are open. It summarizes differences in utilization, migrations, execution, blocking, response time, synchronization activity, and deadline misses.

This is an optional comparison tool. It is not required by the basic investigation workflow. When you use it, compare equivalent workload phases and measurement ranges.

AI Assistant

The optional AI Assistant explains Analysis Findings and Statistics measured by BT-FViewer. It does not replace timeline verification or create measurements that are missing from the trace.

Recommended use:

1. Select a finding or define a time range with cursors.
2. Ask the AI Assistant to investigate or explain it.
3. Review the cited Statistics and timeline evidence.
4. Use **Verify with AI** to challenge the proposed cause.
5. If you make a change, capture a new trace and repeat the same scoped measurements.

Available context levels are **Compact**, **Balanced**, and **Full evidence**. Compact uses fewer tokens, and Balanced is the default. Configure the model, endpoint, authentication, context, privacy, and reply language in **Settings → AI**.

Import `examples/ai/presets.json` for example Ollama, OpenAI, Gemini, DeepSeek, and Grok configurations. Local Ollama does not require an API key. Cloud services may send trace evidence to an external provider; use the anonymization and sensitive-trace settings when appropriate.

See [AI.md](#) for setup, privacy, model options, tools, troubleshooting, CLI testing, and evaluation details.

Export

Output	Desktop	Web
Annotated PNG	Snapshot editor	Snapshot editor
Viewport image	Copy to clipboard	Copy to clipboard
SVG	Save SVG	Save SVG
Perfetto JSON	Export Perfetto	Perfetto
Selected BTF range	Save selection as BTF	Download selected range
Statistics report	CSV or HTML	CSV or HTML

BTF Trace Viewer

Output	Desktop	Web
Trace comparison	CSV or HTML	CSV or HTML

Desktop command line

The Desktop CLI uses the same analysis engine as the graphical application. Set `QT_QPA_PLATFORM=offscreen` when running without a display.

Command	Purpose
<code>info</code>	Print a trace summary
<code>report</code>	Generate a Statistics report
<code>compare</code>	Compare two traces
<code>analyze</code>	Check a trace against a baseline for CI
<code>ai-test</code>	Run AI evidence and validation tests
<code>migrations</code>	Export the migration table as CSV
<code>snapshot</code>	Save a timeline, migration, or metric image
<code>perfetto</code>	Export Chrome Trace JSON
<code>slice</code>	Save a selected timestamp range as a smaller BTF file

```
python builds/btf_viewer.py info trace.btf
python builds/btf_viewer.py report trace.btf -o report.html --format html
python builds/btf_viewer.py compare before.btf after.btf -o diff.html
python builds/btf_viewer.py analyze candidate.btf --baseline baseline.btf
  ↪ --fail-on-regression
python builds/btf_viewer.py snapshot trace.btf -o view.png --view timeline
python builds/btf_viewer.py perfetto trace.btf -o trace.json
python builds/btf_viewer.py slice trace.btf -o window.btf --lo 100000 --hi
  ↪ 500000
```

Run `python builds/btf_viewer.py <command> -h` for all options.

Settings

Open **Settings** from the toolbar or press `Ctrl+,`.

Area	Options
Appearance	Theme, fonts, and colorblind-safe palette
Display	Panels, timeline overlays, CPU budget, and task deadlines
Layout	Label width, row height, zoom density, cursor limit, time precision, and chart sizes
AI	Enablement, context level, privacy, provider, model, authentication, and reply language

Desktop stores settings in `btf_viewer.rc` next to the viewer. Web stores them in browser `localStorage`. Changes are previewed immediately; select **OK** to save or **Cancel** to restore the previous values.

Keyboard and mouse

Common shortcuts

Key	Action
<code>Ctrl+O</code>	Open a file
<code>Ctrl+W</code>	Close the current tab
<code>Ctrl+Tab</code> / <code>Ctrl+Shift+Tab</code>	Move between tabs
<code>Ctrl++</code> / <code>Ctrl+-</code>	Zoom in, or zoom out until Fit
<code>Ctrl+0</code> / <code>F</code>	Fit the complete trace
<code>Ctrl+R</code>	Zoom to the cursor range
<code>Ctrl+F</code> / <code>F3</code> / <code>Shift+F3</code>	Find, next result, or previous result
<code>Ctrl+G</code>	Jump to a timestamp

BTF Trace Viewer

Key	Action
Ctrl+K	Open the command palette
C / Shift+C	Place a cursor or clear all cursors
Ctrl+B	Add a bookmark
A	Add an annotation
Ctrl+S	Open the Snapshot editor
Ctrl+Shift+S	Save SVG
Ctrl+Shift+E	Export Perfetto JSON
?	Show Web shortcuts

Mouse controls

Action	Result
Mouse wheel	Pan
Ctrl+wheel	Zoom
Left-drag the background	Pan
Middle-drag	Zoom to a selected range
Ctrl+left-drag	Measure the time between two points
Click the timeline	Place a cursor
Drag a cursor or mark	Move it
Right-click	Open the context menu

Build and test

Most users can ignore this section.

BTF Trace Viewer

Task	Command
Build Desktop and Web	<code>make -C BTFViewer</code>
Build the Desktop package	<code>make -C BTFViewer bundle</code>
Build the Web application	<code>make -C BTFViewer web</code>
Run the guided demo	<code>make -C BTFViewer demo</code>
Run Desktop tests	<code>make -C BTFViewer test</code>
Run Web tests	<code>make -C BTFViewer test-web</code>
Run all tests	<code>make -C BTFViewer test-all</code>
Build documentation PDFs	<code>make -C BTFViewer doc</code>
Run from source	<code>python -m btf_viewer_pkg trace.btf from BTFViewer/</code>

Edit Desktop sources in `btf_viewer_pkg/` and Web sources in `web/`. Commit regenerated files under `builds/` with the source changes. Shared parser and Statistics results are checked with fixtures under `tests/fixtures/`.

See `TRACE_FORMAT.md` in the parent directory for the BTF field reference.

Contributors

Thanks to everyone who has contributed to this project.

Contributor	Contribution
DiogoRoseira	CPU Load Graph and metric-distribution charts
